Reviewer Pack — Minimal Rank of p-Adic Fourier Coefficients vs Resolution Pr…

Entry d46b2cf3a537 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 10:50:44 UTC

Minimal Rank of p-Adic Fourier Coefficients vs Resolution Proof Tree Diameter over Planar Graphs

Entry ID: `d46b2cf3a537` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 10:50:44 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (Fourier Analysis) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity over Planar Graphs

Statement:

> ['For a planar graph G with n vertices, the resolution proof tree diameter t*(G) is upper-bounded by a function of the p-adic Fourier coefficients of its adjacency matrix A. Specifically, there exists a constant c > 0 such that t*(G) ≤ c * ||A||_p^2, where ||A||_p is the maximum absolute value of the p-adic Fourier coefficients of A.', 'For all planar graphs G with n vertices, the ratio between the resolution proof tree diameter and the square root of the sum of the squares of the p-adic Fourier coefficients of its adjacency matrix satisfies: t*(G) / sqrt(sum(||A||_p^2)) ≤ c.', 'No counterexample exists for a planar graph G with n vertices where t*(G) > c ||A||_p^2.']

Rationale (proposer's reasoning):

> ['The p-adic Fourier coefficients of the adjacency matrix have been previously used to study the spectrum and dynamics of graphs, suggesting potential connections to the resolution proof complexity which measures the size of a proof that refutes a graph as unsatisfiable.', 'This conjecture proposes a direct relationship between these invariants, which if true could reveal deep structural properties of resolution proofs on planar graphs.', 'By linking p-adic Fourier analysis with resolution proof complexity over planar graphs, this conjecture offers a novel perspective that has not been extensively explored.']

Taxonomy category: `resolution_proof_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a0063c531e64c1d5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a planar graph G, if the ratio t*(G) / sqrt(sum(||A||_p^2)) is bounded by a constant c for all n ≤ 40 and across 30 different seeds, then support for the conjecture is provided.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_planar_graph(n):
    if n <= 4:
        # Generate a small planar graph using a known construction for n = 3, 4
        if n == 3:
            return [[0, 1, 1], [1, 0, 1], [1, 1, 0]]
        elif n == 4:
            return [[0, 1, 1, 1], [1, 0, 1, 1], [1, 1, 0, 1], [1, 1, 1, 0]]
    else:
        raise NotImplementedError("Mapping undefined for n > 4")

def p_adic_fourier_coefficients(matrix, p):
    n = len(matrix)
    fourier_coeffs = []
    for i in range(n):
        coeff = 0
        for j in range(n):
            coeff += matrix[i][j] * (p ** (-i - j))
        fourier_coeffs.append(coeff)
    return max(abs(c) for c in fourier_coeffs)

def resolution_proof_tree_diameter(graph):
    n = len(graph)
    # Simplified heuristic to estimate the diameter
    return n

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
```

```
        graph = generate_planar_graph(n)
        p_adic_coeff = p_adic_fourier_coefficients(graph, 2)  # Using prime p=2
        t_star = resolution_proof_tree_diameter(graph)
        if p_adic_coeff == 0:
            continue
        ratio = t_star / math.sqrt(sum(p_adic_coeff**2 for _ in range(n)))
        results.append(ratio)
    if not results:
        return {
            "metric_name": "Ratio",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No valid instances found"
        }
    mean_ratio = sum(results) / len(results)
    return {
        "metric_name": "Ratio",
        "metric_value": mean_ratio,
        "instances_tested": len(results),
        "conjecture_holds": all(r <= 1 for r in results),  # Assuming c=1 for simplicity
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [3, 5, 7, 11, 13, 17, 19, 23, 29,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} st
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE Reason=Insufficient support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d227f54.py", line 76, in <module>
```

```
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d227f54.py", line 47, in run_trial
    graph = generate_planar_graph(n)
            ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d227f54.py", line 26, in generate_plan
    raise NotImplementedError("Mapping undefined for n > 4")
NotImplementedError: Mapping undefined for n > 4
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide evidence for or against the conjecture. | next: Re-run the test with a planar graph generator that can handle n > 4 and ensure all necessary conditions are met to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13404
2	preregistration	ollama_remote	glm4:latest	0	0	5723
3	novelty	ollama_remote	glm4:latest	0	0	4849
4	novelty	ollama_remote	glm4:latest	0	0	5386
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16079
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11442
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9462
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11064
9	judge	ollama_remote	glm4:latest	0	0	10604

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 88012 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d46b2cf3a537.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d46b2cf3a537.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d46b2cf3a537.tar.gz` (if generated)